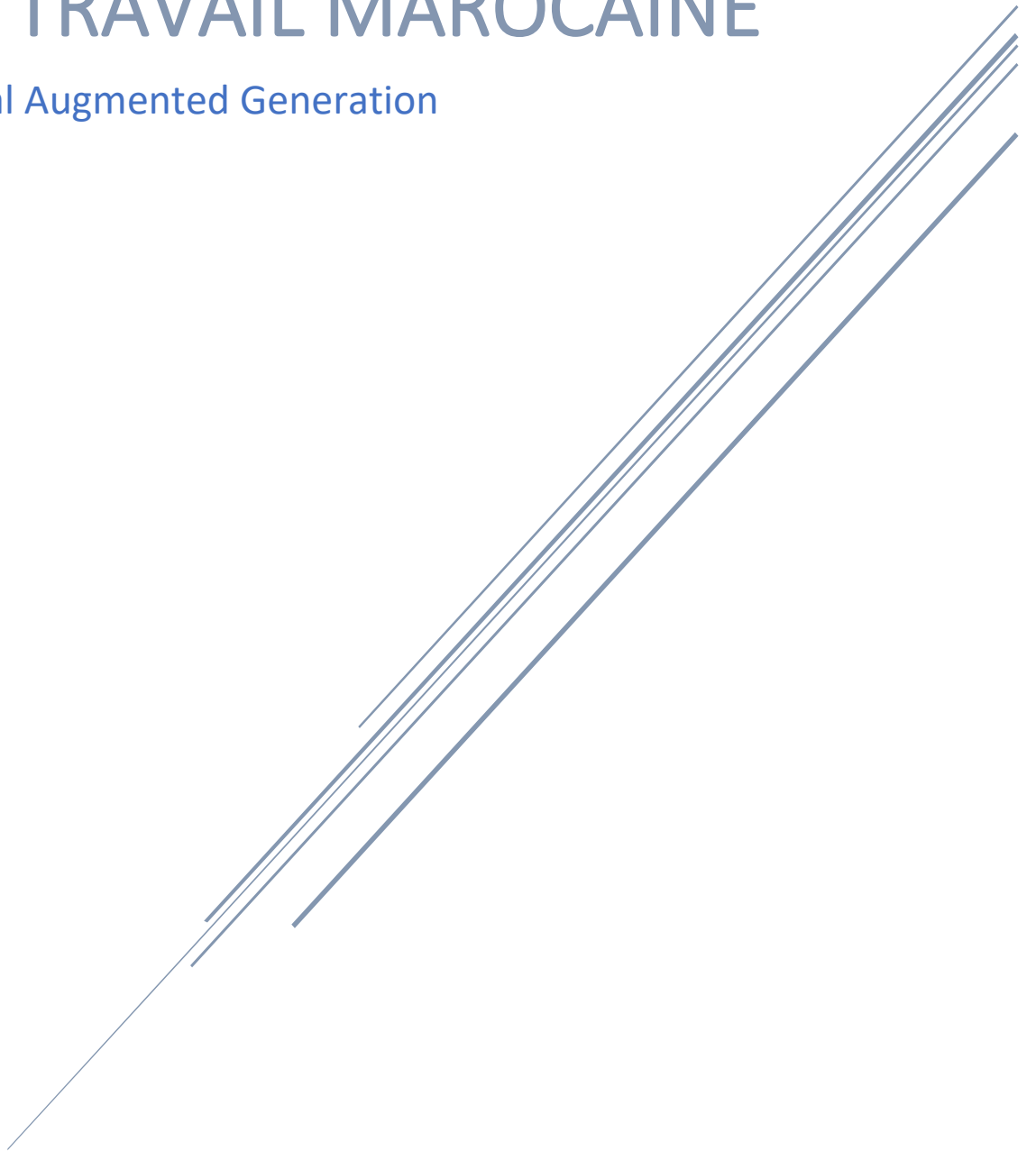


CHATBOT POUR LA LOI DE TRAVAIL MAROCAINE

Retreival Augmented Generation



Réalisé par :
Rajaa LEBNAITI

Table des matières

1) Préparation des données et chunking	5
1-1) Chunking	5
• Regex	5
2) Choix de la base de données et indexation	6
3) Retrieval hybrid and re-ranking	6
1-1) Hybrid Search	6
• Keyword Search :	6
• Semantic Search	6
• Recherche par vecteur	7
2-1) Fusionner les résultats (RRF)	7
3-1) Reranking	7
4) LLM, Prompt engineering et paramètres	7
1-1) Architecture	7
2-1) Les paramètres du modèle	8
3-1) Prompt Engineering & System Prompt	8
• System Prompt	8
• In context learning and chain of thought	8
5) Évaluation, Contrôle des Hallucinations & Observabilité	9
• Gestion des hallucinations	9
• Evaluation de la qualité avec RAGAS	9
• Observabilité avec Phoenix de Arize	9

Liste des tableaux

Tableau 1: Les paramètres du modèle	8
--	---

Liste des figures

Figure 1: Code 1 Chunking using Regex.....	5
Figure 2: Architecture globale du projet AI-Generated	10

Plan de travail

1) Préparation des données et chunking

1-1) Chunking

Il existe plusieurs méthodes pour faire le découpage des documents. Dans ce projet, on va utiliser la méthode de Regex.

Regex

Cette méthode permet de découper un document textuel en des segments (chunks) selon des règles linguistiques du texte (phrases, paragraphes, mots) plutôt que par un simple nombre de caractères ou de mots.

Les morceaux obtenus contiennent des idées complètes et linguistiquement cohérentes, ce qui permet de réduire l'hallucination du modèle.

Pour cela on va utiliser la formule regex suivante :

```
1 pattern = r"(\nArticle\s*(?:premier|\d+))"
2 parts = re.split(pattern, full_text)
3
4 chunks = []
5
6 for i in range(1, len(parts), 2):
7     title = parts[i].strip()
8     content = parts[i+1].strip()
9
10
11
12     full_article_text = f"{title}\n{content}"
13     clean_title = re.sub(r"\s+", " ", title)
14
15
16     chunks.append(Document(
17         page_content=full_article_text,
18         metadata={"article_no": clean_title, "source": "Code du Travail Marocain"}
19     ))
```

Figure 1: Code 1 Chunking using Regex

On obtient : **1chunk = 1 document.**

On a aussi attaché à chaque article des métadonnées :

{ "article_no": "Article 16",

"source": "Code du travail marocain" }

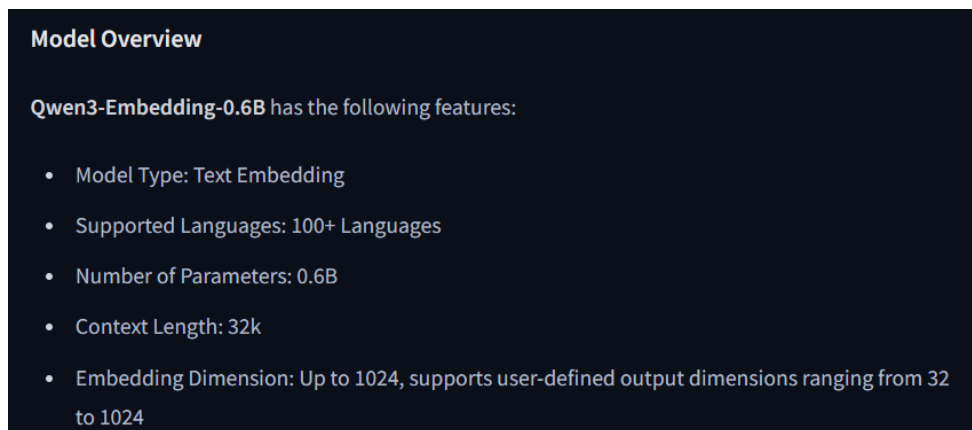
2) Choix de la base de données et indexation

Il existe plusieurs types de base de données vectorielle comme ; ChromaDb et Weaviate.

Pour ce projet on va utiliser ChromaDb qui permet de gérer à la fois les vecteurs et le filtrage par métadonnées. Elle s'installe en local par `pip install chromadb`

Pour la partie embedding, on va utiliser Qwen3-Embedding :0.6B

(Hugging Face, s.d.)



3) Retrieval hybrid and re-ranking

1-1) Hybrid Search

Dans ce projet, on va adopter une approche hybride de recherche.

On va utiliser à la fois la recherche par mots (**keyword search**) et aussi recherche par sens (**semantic search**).

🚦 Keyword Search :

Ici on va utiliser la méthode **BM25** au lieu de la méthode **TF-IDF** vu que cette dernière est obsolète et moins performante sur la pertinence des fréquences de mots

🚦 Semantic Search

On va utiliser un BI-Encoder, et surtout on va utiliser le modèle multilingue **araphrase-multilingual-MiniLM-L12-v2** pour pouvoir travailler avec du texte français/arabe.

Recherche par vecteur

Ici on va utiliser ANN (***Approximate Nearest Neighbors***) comme l'algorithme HNSW qui est déjà intégré par défaut dans ChromaDB, vu que c'est plus rapide que d'utiliser l'algorithme KNN (***K- Nearest Neighbors***) même si ce dernier engendre une recherche exacte mais trop lente sur de gros volumes.

2-1) Fusionner les résultats (RRF)

Vu qu'on va travailler avec une approche hybride, alors le BM25 va donner une liste de documents pertinents ainsi que le Bi-encoder (recherche sémantique) va aussi donner une autre liste.

C'est pourquoi on va utiliser le RRF (***Reciprocal Rank Fusion***) qui permet de fusionner ces deux listes en un score unifié. Cela garantit que si un article contient le mot-clé ET est sémantiquement proche, il remonte en haut.

3-1) Reranking

On va utiliser un Cross-Encoder (comme un modèle ***Cohere Rerank*** ou un modèle open-source léger de chez Hugging Face) pour évaluer la liste des documents récupérer avant de l'envoyer au LLM.

Pourquoi ? Le Bi-encoder calcule les vecteurs séparément (rapide mais perd de la précision relationnelle). Le Cross-Encoder prend la Requête + le Document en même temps pour calculer un score d'une précision extrême. On passe par exemple les 10 meilleurs chunks issus de la fusion RRF dans le Cross-Encoder, et on ne garde que les 3 meilleurs.

4) LLM, Prompt engineering et paramètres

1-1) Architecture

L'architecture qu'on va adopter est un simple RAG (sans agentic rag).

C'est un pipeline linéaire où on a :

Requête → Retrieval → Generation

Pour le modèle LLM on peut utiliser modèle local via Ollama (Llama 3 ou Mistral) ou l'API d'OpenAI (gpt-4o-mini qui est extrêmement abordable et précis)

2-1) Les paramètres du modèle

Pour un chatbot juridique, on a besoin d'un comportement qui soit déterministe (pas de créativité). Pour ce faire, on va adopter les paramètres suivants :

Paramètre	Valeur	Justification
Température	0.0 ou 0.1 (proche de 0)	Le modèle accentue les mots à fort probabilités
Top P	0.9	Permet de garder une structure grammaticale fluide
Repetition penalty	1.1 à 1.2	Évite que le modèle ne boucle sur les formulations juridiques répétitives.

Tableau 1: Les paramètres du modèle

3-1) Prompt Engineering & System Prompt

🚦 System Prompt

On va utiliser le system prompt pour ajouter des contraintes comme :

- Le modèle ne doit répondre que en se basant sur les documents fournis.
- Si la réponse n'est pas dans les documents, le modèle dis poliment qu'il ne sait pas.
- Le modèle doit citer ses sources.

🚦 In context learning and chain of thought

Dans la prompt, on va donner des exemples en utilisant la méthode « **few shot** » cela permet au modèle de savoir comment décomposer un problème.

5) Évaluation, Contrôle des Hallucinations & Observabilité

Après avoir implémenter tout cela, on doit évaluer le projet.

✚ Gestion des hallucinations

On a ajouté une étape de **Query Rewriting** qui utilise un LLM rapide pour réécrire la question mal formée de l'utilisateur. Ce qui améliore le retrieval.

✚ Evaluation de la qualité avec RAGAS

On va préparer un petit Dataset de test (10-215 questions clés avec les réponses attendues et les vrais articles associés).

Puis on va utiliser RAGAS pour évaluer le RAG sur 3 piliers :

- **Faithfulness** : Est-ce que la réponse générée est entièrement basée sur le context.
- **Response Relevance** : Est-ce que la réponse répond vraiment à la question de l'employé.
- **Context Recall** : Est-ce que le moteur de recherche a bien réussi à retrouver les bons articles de loi.

✚ Observabilité avec Phoenix de Arize

On va utiliser Phoenix, c'est un outil open-source pour visualiser tes traces de RAG. On peut voir exactement ce que l'utilisateur a tapé, quel chunk a été récupéré par ChromaDB, comment le prompt a été formaté, et ce que le LLM a répondu.

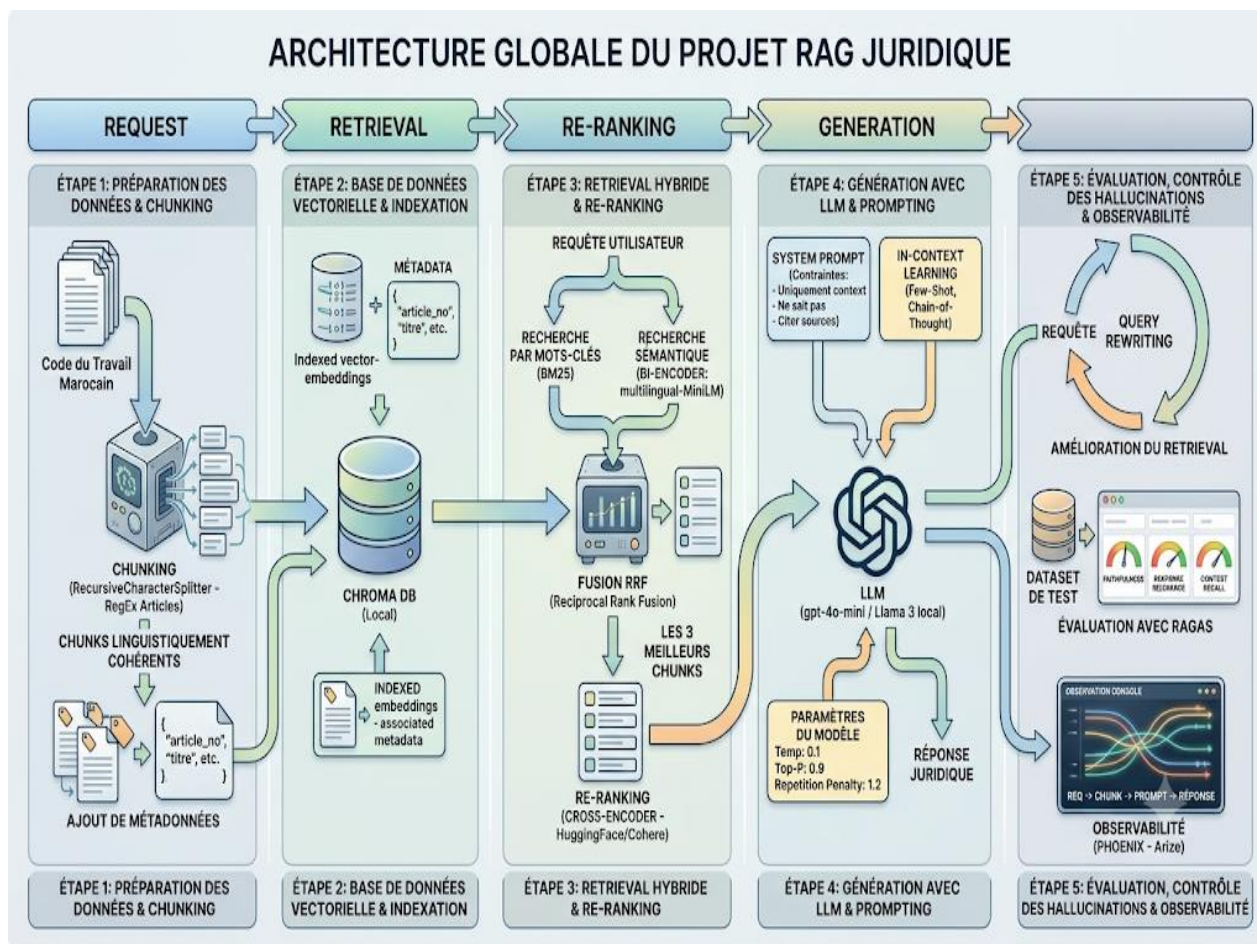


Figure 2: Architecture globale du projet AI-Generated

ARCHITECTURE GLOBALE DU CHATBOT JURIDIQUE (RAG)

Pipeline de bout en bout

